

Modern App Architecture

Docker, Redis, and PostgreSQL JSON

4-Hour Bachelor Level Deep-Dive

Lecture Agenda

A comprehensive 4-hour technical journey

- Hour 1: Containerization with Docker & Compose
- Hour 2: In-Memory Key-Value Stores (Redis)
- Hour 3: Unstructured Data in SQL (JSON & PostgreSQL)
- Hour 4: Practical Lab & Environment Assembly

The Architecture Evolution

Before diving into specific tools, we must understand why they exist.

- The Monolith: Historically, entire apps were deployed as a single codebase.
- Scaling Issues: Monoliths are difficult to scale. Heavy database traffic requires cloning the whole app.
- Microservices: Modern architecture splits the app into small, independent services.
- The Requirement: Microservices require isolated, lightweight environments.



Hour 1: Containerization

Mastering Docker & Microservice Environments

The Pre-Docker World: VMs

Before containers, we used Virtual Machines to isolate applications on bare-metal servers.

- Hardware Virtualization: VMs simulate physical hardware using a Hypervisor.
- Guest OS Requirement: Every VM requires a full, heavy operating system to run.
- Resource Intensive: Consumes massive amounts of RAM and CPU overhead.
- Slow Boot Times: Takes minutes to boot.



VMs vs. Container Architecture

Virtual Machines

Stack: App -> Libs -> Guest OS -> Hypervisor -> Server.

The Hypervisor translates instructions to the physical hardware. The heavy Guest OS creates a strict, inflexible boundary, wasting gigabytes of space per app.

Docker Containers

Stack: App -> Libs -> Docker Engine -> Server.

Docker virtualizes the Operating System, not the hardware. Containers share the host system's OS kernel but run in isolated user spaces.

Introduction to Docker Core Concepts

- Dockerfile: A text document containing all the commands to assemble an image. The blueprint.
- Docker Image: A read-only, immutable template with instructions for creating a container.
- Docker Container: A runnable instance of an image. You can create, start, stop, or delete a container.

The Developer's Dilemma

"It works on my machine!"

The Ultimate Fix:

Before Docker, code that ran perfectly on a developer's Mac might crash on the Linux production server due to mismatched Node.js versions or missing C++ libraries.

Docker guarantees environment consistency. Because the application is packaged with its dependencies, it behaves identically everywhere.

Evaluating Docker

Advantages:

- Speed: Boots in milliseconds.
- Density: Run 100 containers where you could only run 5 VMs.
- Portability: Run anywhere.
- CI/CD Integration: Perfect for automated pipelines.

Disadvantages & Limits:

- Learning Curve: High barrier to entry.
- Security: Kernel sharing means potential host compromise on exploits.
- Not for Monoliths: Rarely yields benefits for massive legacy apps.

The Data Problem: Volumes

By default, containers are ephemeral. If a container stops or is deleted, all data written inside it vanishes forever. This is disastrous for databases.

- Ephemeral Layer: The writable layer attached to a running container. Tied to the container lifecycle.
- Docker Volumes: Managed by Docker on the host filesystem. They persist independently of container lifecycles. Best for DB data.
- Bind Mounts: Maps a specific file or directory on the host machine directly into the container.

Anatomy of a Dockerfile

How it builds:

Docker reads top-down. Each command creates a cached 'Layer'.

If you change your code (Step 5), Docker reuses the cache for Steps 1-4, making builds blazingly fast.

```
# 1. Base Image
FROM node:18-alpine

# 2. Set directory
WORKDIR /app

# 3. Copy dependencies
COPY package.json ./

# 4. Install
RUN npm install

# 5. Copy code
COPY . .

# 6. Run
CMD ["npm", "start"]
```

Docker Compose

Orchestrating Multi-Container Applications

Introduction to Compose

Docker Compose allows us to define and run multi-container applications using a single, declarative file: `docker-compose.yml`

```
$ docker run -d --name db -p 5432:5432 postgres
$ docker run -d --name cache -p 6379:6379 redis

# We simply write a YAML file once and run:
$ docker compose up -d
```

Anatomy of docker-compose.yml

The Keys:

- services: The root key.
- image: Pulls a pre-built image from Docker Hub.
- environment: Passes variables to the container.
- ports: Maps HostPort:ContainerPort.

```
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"
```

Managing Environment Variables

Method 1: Hardcoding (Dev)

Good for quick local labs, but bad for security if committed to version control.

services:

postgres:

image: postgres:15

environment:

POSTGRES_USER: admin

POSTGRES_PASSWORD: secret

Method 2: env_file (Production)

The industry best practice. Store secrets in a separate .env file that is ignored by Git.

services:

postgres:

image: postgres:15

env_file:

- .env

Compose Networking

How do containers talk to each other without static IPs? Compose creates a Default Bridge network. Containers discover each other using their service names as hostnames.

```
services:
  api:
    image: my-node-app
    networks:
      - backend-net

networks:
  backend-net:
    driver: bridge
```


Docker Clean Slate Commands

Sometimes things break, ports are stuck, or you run out of disk space. Here is how to aggressively reset your Docker environment.

```
# Stop & Remove Compose Stack
```

```
$ docker compose down -v
```

```
# Remove unused networks
```

```
$ docker network prune
```

```
# THE NUCLEAR OPTION: Removes all stopped containers & images
```

```
$ docker system prune -a --volumes
```

Hour 2: In-Memory Key-Value

Entering the World of Redis & NoSQL

Introduction to NoSQL

Why NoSQL?

Relational databases (SQL) are rigid. They require predefined schemas and struggle to scale horizontally across multiple physical servers.

NoSQL emerged to handle massive, unstructured datasets and extreme traffic loads.

Families of NoSQL

- Document Stores: MongoDB
- Column-Family: Cassandra
- Graph Databases: Neo4j
- Key-Value Stores: Redis, DynamoDB

The Key-Value Store Concept

Functions identically to a Dictionary in Python or a HashMap in Java. Data is stored as pairs, mapping a unique key to a value.

Time Complexity = $O(1)$

Constant Time Lookup:

Fetching data via an exact key is instantaneous, whether the database has ten records or ten billion records.

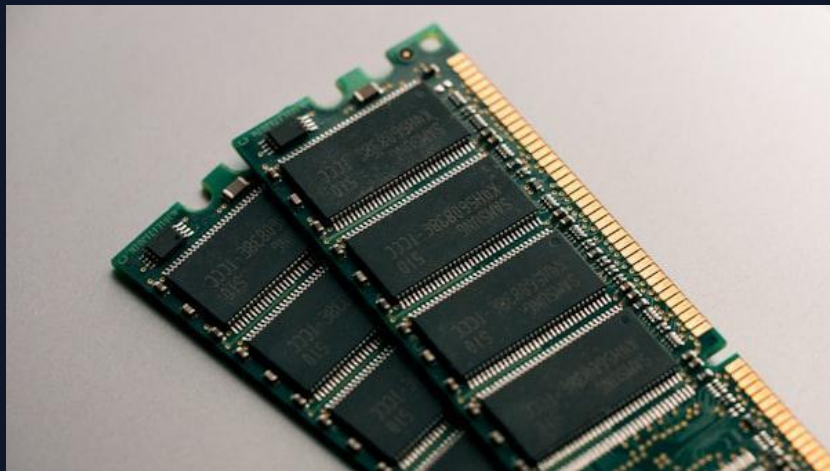
What is Redis?

Redis (Remote Dictionary Server) is an open-source key-value store.

The In-Memory Magic:

Unlike traditional databases that write to physical disks, Redis operates entirely in RAM.

Because RAM is exponentially faster, Redis can handle millions of operations per second with sub-millisecond latency.



Redis Data Structures

Redis isn't just plain strings; it offers complex data structures mapped to keys.

- **Strings:** The most basic type. Can store text, serialized JSON, or even binary data up to 512MB.
- **Lists:** Linked lists of strings. Excellent for implementing queues, stacks, or retaining the latest N items.
- **Hashes:** A collection of field-value pairs. Perfect for representing objects (e.g., a User with name, age, email fields).
- **Sorted Sets:** Ordered lists automatically ranked by a numeric score.

The Synergy: SQL + Redis

RDBMS (The Anchor)

- System of Record.
- Long-term, permanent disk storage.
- Maintains relational integrity.
- Slower (10-50ms query times).

Redis (The Shield)

- Caching Layer.
- Volatile, high-speed RAM storage.
- Absorbs repetitive reads.
- Blazing fast (<1ms query times).

The Caching Flow

How the application interacts with both databases during a read request:

1. App Request: User requests the "Top 10 News Articles" page.
2. Query Redis: App checks Redis first.
 - Cache Hit: Return instantly.
 - Cache Miss: Proceed to step 3.
3. Query SQL: App executes expensive SQL JOINS to compute the top articles.
4. Store & Return: App saves result in Redis for future users, then returns data to the user.

Redis Real-World Scenarios

- Session / Auth Management: Storing user sessions or JWTs with an expiration time (TTL). Fast lookup on every API request.
- Rate Limiting: Using counters in Redis to restrict users to "100 API requests per minute" to prevent DDoS attacks.
- Real-time Leaderboards: Using Sorted Sets to automatically rank millions of player scores in gaming without complex SQL queries.

Hour 3: JSON Foundation & Postgres

Bridging the gap between Strict SQL and Flexible Data

What is JSON?

JSON is a lightweight data-interchange format. It is built on two structures: a collection of name/value pairs (Objects) and ordered lists (Arrays).

```
// This is a simple JSON Object
{
  "key": "value",
  "language": "Agnostic"
}
```

JSON Data Types: Primitives

At its core, a JSON object maps a String key to a value. Strict Rule: Keys **MUST** be wrapped in double quotes.

```
{  
  // 1. String (Must be double quotes)  
  "username": "jdoe99",  
  
  // 2. Number (Integer or Float, no quotes)  
  "age": 25,  
  
  // 3. Boolean (true or false, lowercase, no quotes)  
  "is_student": true,  
  
  // 4. Null (Represents empty/missing data)  
  "graduation_date": null  
}
```

JSON Data Types: Nested Objects

Values can also be complex types. An object can contain another object, allowing for deep, hierarchical nesting.

```
{  
  "product_name": "Gaming Laptop",  
  "specifications": {  
    "cpu": "Intel Core i9",  
    "ram": "32GB",  
    "dimensions": {  
      "width_cm": 35.9  
    }  
  }  
}
```

JSON Data Types: Arrays

An Array is an ordered list of values enclosed in square brackets []. They are used for collections of items.

```
{  
  "user_id": 105,  
  "tags": ["premium", "subscriber", "beta-tester"],  
  "recent_scores": [85, 92, 78, 99],  
  "mixed_bad_practice": [1, "apple", true]  
}
```

The Industry Standard: Arrays of Objects

The most common payload you will see in modern REST APIs is a JSON Array containing JSON Objects. This represents a list of records.

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "role": "Admin"  
  },  
  {  
    "id": 2,  
    "name": "Bob",  
    "role": "User"  
  }  
]
```

The Unstructured Data Problem

The Relational Constraint

Imagine an E-Commerce store. A T-Shirt has: Size, Color. A Laptop has: RAM, CPU, Screen.

In standard SQL, you must create sparse columns (many NULLs) or complex EAV tables to handle dynamic attributes.

The Postgres Solution

Instead of migrating entirely to NoSQL (MongoDB), PostgreSQL introduced native JSON support.

This allows strictness for critical data (Users, Payments) alongside schema-less JSON flexibility in the same database.

JSON vs. JSONB (Binary)

'JSON' Type

Stores data as an exact text copy.
Preserves whitespace.

Slow, because PostgreSQL must re-parse the entire text string on every query.

'JSONB' Type

Stores data in a decomposed binary format. Strips whitespaces and duplicate keys.

JSONB is drastically faster to process and supports advanced GIN indexing. This is the industry standard.

The GIN Index Advantage

Standard SQL B-Tree indexes fail at searching inside complex objects.

Generalized Inverted Index (GIN)

It creates an index entry for every key and value inside the JSONB object, allowing PostgreSQL to instantly locate records that contain specific nested JSON structures.



JSONB Syntax: DDL & Insert

1. Table Creation

We define the column simply as JSONB.

2. Inserting Nested Data

We insert it as a standard formatted string. Postgres validates the JSON upon insert.

```
CREATE TABLE products (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    attributes JSONB  
);  
  
INSERT INTO products  
(name, attributes) VALUES (  
    'Laptop',  
    '{"ram": "16GB"}'  
);
```

JSONB Syntax: Query & Filter

3. Extracting Data

Use `->>` to extract as Text, `->` to extract as JSON object.

4. Filtering Data

Use `@>` (Containment) to check if JSON contains a subset. This utilizes the GIN Index!

```
-- Returns "16GB" as text
SELECT attributes->>'ram'
FROM products;
```

```
-- Containment Filter
SELECT name FROM products
WHERE attributes @>
'{"ram": "16GB"}';
```

Hour 4: Practical Lab

Environment Assembly & Execution

Native Linux Installation

For Debian/Ubuntu systems, we can install directly to the host OS using the apt package manager. (Note: Docker is preferred over native DB installs).

```
$ sudo apt update
$ sudo apt install -y docker.io docker-compose-v2
$ sudo usermod -aG docker $USER

# Install Databases Native (Optional for this lab)
$ sudo apt install -y redis-server postgresql
```

Lab Setup: docker-compose.yml

Create a directory and save the declarative structure we studied earlier.

```
services:
  postgres:
    image: postgres:15
    container_name: lab_postgres
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secretpassword
      POSTGRES_DB: app_db
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    container_name: lab_redis
    ports:
      - "6379:6379"
```

Lab Execution - Step 1 & 2

1. Spin Up Compose Stack
2. Access Postgres interactively

```
$ docker compose up -d or sudo docker compose up -d
$ docker ps

$ docker exec -it lab_postgres psql -U admin -d app_db
# CREATE TABLE products (id SERIAL, attributes JSONB);
# \q (to exit)
```


Lab Execution - Step 3 & 4

3. Access Redis

4. Simulate Cache TTL and observe expiration after 10 seconds

```
$ sudo docker start lab_redis
$ sudo docker exec -it lab_redis redis-cli

$ SETEX laptop:ram 10 "16GB"
$ GET laptop:ram

# Wait 10 seconds...
$ GET laptop:ram # Returns (nil)

# exit
```

Teardown & Cleanup

Developers should spin down environments they aren't actively using to save battery and RAM on their local machines.

```
# Stopping Containers (Pauses CPU, saves data)
```

```
$ docker compose stop
```

```
# Complete Teardown (Destroys containers and networks)
```

```
$ docker compose down
```

```
# Add -v to also delete persistent data volumes
```

```
$ docker compose down -v
```

Reviewing Architecture

We utilize Docker for consistent delivery.

We utilize PostgreSQL for permanent relational & JSON storage.

We utilize Redis for volatile, high-speed RAM caching.

Questions?

THANK YOU